COMPILER DESIGN · PROJECT REVIEW 1

Tether

A Source-Mapped Transpiler for Debuggable High-Level Code

25% Implementation Report and Viva Preparation

Student	Antra
Registration number	24BDS0066
Subject	Compiler Design
Review	Review 1: 25% implementation (milestones 1–2 of 8)
Compiler version	0.2.0 (front end: lexer, parser, diagnostics, CLI, playground)
Reference document	Tether PRD v1.0, 25 September 2026 (docs/Tether-PRD.pdf)
Date	9 October 2026

2 / 8 milestones complete (25%)	**63 / 63** automated tests passing	**956** lines of compiler source (src/)	**4 / 4** benchmark programs lex and parse	**0** third-party dependencies

How to read this document. Sections 1–9 describe what has been built for the 25% review, with evidence. Section 10 maps every requirement in the PRD to its status. Section 11 describes how the remaining 75% will be built. Section 12 is a bank of likely viva questions with prepared answers, covering both the finished work and the work still to come. Appendix A is a five-minute live demo script.

Contents

1. Project overview 4

2. Scope of the 25% milestone 5

2.1 Deliverables completed for this review 5

2.2 Compiler pipeline: built vs planned 6

3. Architecture and code layout 6

3.1 Data flow 7

3.2 Design principles applied 7

4. Source positions (spans) 7

4.1 Conventions 8

4.2 The span invariant (FR-P2) 8

5. Lexical analysis 8

5.1 Key techniques 9

5.2 Sample output: --dump-tokens 9

5.3 Lexical errors 9

6. The frozen grammar 10

6.1 Properties of the grammar 11

7. Syntax analysis 11

7.1 Precedence climbing 11

7.2 The abstract syntax tree 12

7.3 Error recovery: panic mode at statement granularity (FR-P3) 12

7.4 Avoiding cascading errors 13

8. Diagnostics and command-line interface 13

8.1 Caret diagnostics (FR-S3) 13

8.2 Command line (FR-C1 – FR-C3) 13

9. The playground web page 15

9.1 How it is served 16

10. Testing, verification and requirements status 17

10.1 Test suite 17

10.2 Golden-file testing 17

10.3 Measured performance 17

10.4 Requirements traceability 18

10.5 Known limitations at this stage 19

11. Plan for the remaining 75% 19

11.1 Milestone 3: semantic analysis . . . 19
11.2 Milestone 4: lowering and a naive JavaScript emitter . . . 20
11.3 Milestones 5–6: Base64 VLQ and the source map . . . 20
11.4 Milestones 7–8: browser proof and optimisation . . . 21
11.5 Risks and decisions log . . . 22
12. Viva preparation: likely questions and answers . . . 23
A. The project and its scope . . . 23
B. Lexical analysis . . . 23
C. Grammar and parsing . . . 24
D. Positions and spans . . . 25
E. Error handling and diagnostics . . . 26
F. Testing, tooling and the playground . . . 26
G. Semantic analysis (milestone 3, planned) . . . 27
H. Lowering and code generation (milestone 4, planned) . . . 28
I. Source maps (milestones 5–7, planned) . . . 28
J. Optimisation (milestone 8, planned) . . . 29
K. General compiler theory applied to Tether . . . 29
Appendix A. Five-minute live demo script . . . 31
Appendix B. Glossary . . . 32

1. Project overview

Tether compiles a small, statically typed imperative language (files ending in `.tth`) to ECMAScript 2015 JavaScript. Alongside the `.js` file it will emit a **Source Map v3** file, so that an ordinary JavaScript debugger (Chrome DevTools, Node) shows breakpoints, stack traces and variable names in terms of the original Tether source, not the generated JavaScript.

The engineering premise is that **source position is a first-class artifact**. A position is not something to print in an error message and throw away; it must survive lexing, parsing, semantic analysis, lowering and code generation without loss, because the source map at the end is nothing more than a table of *generated position → original position*. If any phase drops or shifts a position, the map still loads but silently points to the wrong place. That is why every token and every tree node built so far carries an exact span.

1.1 The language at a glance

Three types (`int`, `bool`, `string`), `let` bindings with inferred or annotated types, top-level functions with typed parameters and recursion, `if`/`else`, `while`, `for`, `return`, `print`, and the usual arithmetic, comparison and logical operators. No closures, arrays, objects or modules (PRD section 11). One of the four benchmark programs:

```
// Fibonacci, computed recursively and iteratively.        (examples/fib.tth)

fn fib(n: int): int {
  if (n < 2) {
    return n;
  }
  return fib(n - 1) + fib(n - 2);
}

fn fibIter(n: int): int {
  let a = 0;
  let b = 1;
  for (let i = 0; i < n; i += 1) {
    let next = a + b;
    a = b;
    b = next;
  }
  return a;
}

for (let i = 0; i <= 15; i += 1) {
  print(fib(i));
}
print(fibIter(40));
```

1.2 Goals (from the PRD)

ID	Goal	Where it stands at 25%
G1	Compile a frozen, documented language subset to JS that runs on Node 18+ and browsers	Grammar frozen and documented; JS output arrives in milestone 4
G2	Carry a (line, column, length) span from token through AST, lowering and emission	Done for tokens and AST; invariant tested on every node
G3	Emit a Source Map v3 file an off-the-shelf debugger accepts	Milestones 5–7
G4	Report several diagnostics per run, each anchored to a precise span	Done for lexical and syntax errors (7 errors in one run)
G5	Measure positional fidelity lost under optimisation	Milestone 8 (Could priority)

2. Scope of the 25% milestone

The PRD (section 9) splits the work into eight weekly milestones, each with an exit criterion that can be checked. Milestones 1 and 2, the complete compiler **front end**, are finished, which is two of eight, or 25%. Two pieces from later milestones were also built early because the front end needed them: caret diagnostics (milestone 3) and the full command-line interface skeleton.



2 of 8 milestones complete = 25%. Week 3 has started (caret diagnostics already done).

Week	Milestone	Exit criterion (PRD)	Status	Evidence
1	Grammar frozen in BNF; scanner with spans	`--dump-tokens` prints a span for every token	Done	`docs/grammar.md`; golden `*.tokens` files for all 4 benchmarks
2	Recursive-descent parser and AST	`--dump-ast` prints spans; benchmarks parse	Done	Golden `*.ast` files; span-invariant test over every node
3	Symbol table, type checking, caret diagnostics	AC-4 passes	Partial	Caret rendering done and tested; checker not started
4	Lowering pass; naive JS emitter (no map)	AC-1 passes	Planned	`--dump-core` reserved in CLI
5	Base64 VLQ encoder and mapping table	Encoder passes published vectors	Planned	`--dump-mappings` reserved
6	Source map emission; reference-decoder validation	AC-3 passes	Planned	`toMapPosition()` boundary already in place
7	Browser verification; screenshots; stack traces	AC-2 and AC-5 pass	Planned	—
8	-01 and fidelity measurement; final report	AC-6 recorded	Planned	-01 flag already accepted

2.1 Deliverables completed for this review

- **Frozen grammar** in BNF with a precedence table and context rules (`docs/grammar.md`).
- **Position conventions** documented (`docs/positions.md`): 1-based internally, UTF-16 columns, tab = 1 column, a single 0-based conversion point for the source map.
- **Lexer** (`src/lexer.js`): every token carries a span; five kinds of lexical error, each with a precisely placed caret; scanning continues after errors.
- **Parser** (`src/parser.js`): hand-written recursive descent with precedence climbing; 21 AST node kinds; every node carries a span; panic-mode recovery reporting up to 20 errors per run.
- **Diagnostics** (`src/diagnostics.js`): `file:line:col: error: message`, the source line, and carets under the exact span, sorted in source order.
- **CLI** (`bin/tether.js`, `src/cli.js`): `tether build <file>` with every PRD flag accepted, `--dump-tokens`, `--dump-ast`, and exit codes 0/1/2.
- **Playground web page** (`playground/index.html`): runs the real compiler modules in the browser and shows tokens, the syntax tree and diagnostics live, with span highlighting.
- **63 automated tests**, including golden-file tests, a property test and a determinism test. All pass on Node 18+.

2.2 Compiler pipeline: built vs planned

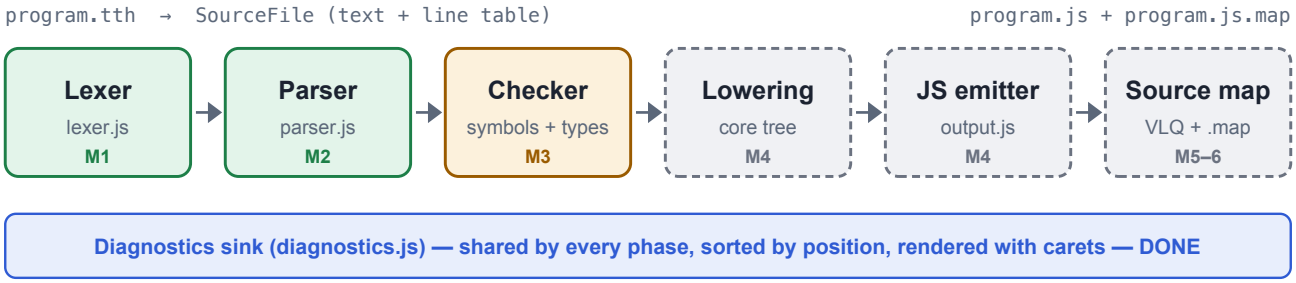


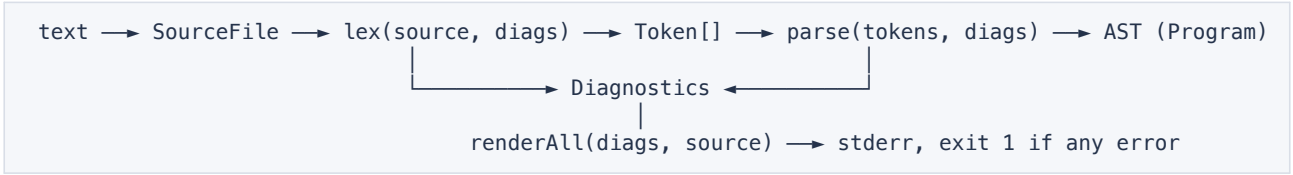
Figure 1. The six phases. The two phases on the left are complete; the diagnostics sink that every phase reports into is also complete.

3. Architecture and code layout

Each phase is a separate ES module with one job, so each can be tested on its own (NFR-6). `src/compiler.js` only wires them together. Nothing in `src/` except `cli.js` touches Node APIs, which is what lets the playground load the same modules in a browser. The project has **no dependencies** at all: no parser generator, no test framework beyond Node's built-in `node:test` (NFR-1).

File	Lines	Responsibility
`src/span.js`	64	Span type; `union()`, `spanAfter()`, `formatSpan()`; `toMapPosition()`, the only 1-based → 0-based conversion
`src/source.js`	28	`SourceFile`: source text plus a line-start table; returns the text of any line for diagnostics
`src/lexer.js`	180	Scanner: text → token array ending in EOF; lexical error reporting
`src/parser.js`	383	Recursive descent + precedence climbing; AST construction with spans; panic-mode recovery
`src/ast.js`	52	`CHILD_FIELDS` schema for all 21 node kinds; generic `children()` and `walk()`
`src/diagnostics.js`	65	Collects errors and notes; renders them with caret lines; sorts by position
`src/dump.js`	51	Stable text formats for `--dump-tokens` and `--dump-ast` (compared byte-for-byte by tests)
`src/compiler.js`	17	Pipeline wiring: `frontEnd(name, text)` → `{ source, diags, tokens, ast }`
`src/cli.js`	116	Argument parsing, exit codes, I/O injection for tests
`bin/tether.js`	4	Executable entry point
`playground/index.html`	621	Browser workbench that imports `src/` directly
`scripts/serve.js`	65	Local static file server for the playground (localhost only)
`test/*.test.js`	547	63 tests across 6 files, plus golden files and error fixtures

3.1 Data flow



A single `Diagnostics` object is passed to every phase. The lexer and parser never print anything; they only record diagnostics with spans. The CLI decides how to render them and what exit code to return. The parser runs even when the lexer found errors, so one run reports as much as possible (goal G4).

3.2 Design principles applied

- **One phase, one module, one test file** (NFR-6). A failing test points at one phase.
- **One write path for positions.** The lexer moves forward only through `advance()`; positions become 0-based only in `toMapPosition()`. The same idea will be used for the emitter (FR-G2).
- **Schema-driven tree walking.** `CHILD_FIELDS` lists each node kind's children in source order. The dumper, the playground and the span-invariant test all walk the tree through it, so adding a node kind is a one-line change.
- **Determinism** (NFR-2). No timestamps, randomness or hash-order dependence in output; a test compiles each benchmark twice and compares.
- **Injectable I/O.** `main(argv, io)` takes `stdout`, `stderr` and `readFile` functions, so CLI tests run in-process without spawning Node.

4. Source positions (spans)

Every token and every AST node has a `span` object. This is the foundation the source map will be built on.

Field	Base	Meaning
`file`	—	Source file name exactly as given on the command line
`offset`	0-based	Index of the first UTF-16 code unit in the file text (used for slicing and highlighting)
`length`	—	Number of UTF-16 code units covered
`line`, `col`	1-based	Start position, as shown to the user
`endLine`, `endCol`	1-based	Position just past the last code unit (needed for nodes that span lines)

4.1 Conventions

Rule	Decision	Reason
Column unit	UTF-16 code units	JavaScript strings and Source Map v3 consumers count this way. An emoji in a string counts as 2 columns.
Tabs	A tab advances the column by 1 (FR-L5)	Source-map columns are plain code-unit indices, and Chrome DevTools resolves them so. This answers open question 3 in the PRD.
Line ends	`\n`, `\r\n` and a lone `\r` each end one line	Files written on Windows, Unix and old Mac all give the same line numbers.
Base	1-based everywhere inside the compiler	Matches what editors and diagnostics show.
Conversion	Only in `toMapPosition()`	Off-by-one between 1-based and 0-based is rated a High risk in the PRD (section 10). One function, covered by a property test, removes it.

```
// src/span.js — the 1-based to 0-based boundary (FR-L2).
// The source-map writer must obtain every original-position field through this function.
export function toMapPosition(s) {
  return { line: s.line - 1, column: s.col - 1 };
}

// The smallest span that covers both a and b. Either argument may be null.
export function union(a, b) { ... }

// A zero-length span just past the end of s: where a missing ';' belongs.
export function spanAfter(s) { ... }
```

4.2 The span invariant (FR-P2)

A leaf node's span is exactly its token's span. An interior node's span is the union of its children's spans and of any tokens it consumed directly. For example, a `Let` statement runs from `let` through the closing `;`, and an `If` runs from `if` to the end of its last block. A test walks **every node in all four benchmarks** and checks that children lie inside their parent, appear in source order without overlapping, and that every leaf has the span of a real token.

5. Lexical analysis

The scanner in `src/lexer.js` is hand-written (NFR-1) and makes a single left-to-right pass over the text. It looks at the current character and dispatches: whitespace, a `//` comment, a digit, an identifier start, a double quote, a punctuator, or otherwise an error. Each branch is in effect a small hand-coded finite automaton.

Token kind	Examples	Notes
`Int`	`0`, `10000`	`value` holds the number; must be ≤ 2^53 − 1
`String`	`"Fizz"`	`value` holds the decoded text; escapes `\n \t \" \\`; may not span lines
`Bool`	`true`, `false`	Scanned as a word, then turned into a literal with a boolean `value`
`Ident`	`fib`, `bestLength`, `_x1`	Letter or _, then letters, digits, _; ASCII only
`Keyword`	`fn let if else while for return print int bool string`	11 reserved words (plus `true`/`false` as Bool)

Token kind	Examples	Notes
`Punct`	`&& || == != <= >= += -= *= /= %= + - * / % ! = < > ( ) { } , ; :`	27 operators and separators; longest match first
`Error`	#, a lone &	An unknown character; already reported, flagged `reported: true`
`EOF`	—	Always last; zero-length span just past the final character

Each token is `{ kind, lexeme, value, span, reported }`. The `lexeme` is the exact source text; the `value` is the decoded literal value.

5.1 Key techniques

- **Maximal munch.** Identifiers and numbers consume every valid character, so `iffy` is one identifier, not `if` + `fy`. Punctuators are tried longest first, so `<=` wins over `<` and `+=` over `+`.
- **Keyword lookup after scanning.** An identifier-shaped word is checked against the `KEYWORDS` set after it has been scanned, which is simpler and faster than encoding keywords into the automaton.
- **One place that moves forward.** `advance()` is the only code that increments the offset, and it updates line and column at the same time, so the three can never drift apart.
- **Negative numbers are not tokens.** `-5` is a unary minus applied to `5`. This keeps the lexer free of context (`a-5` and `a - 5` lex the same).
- **Linear time.** Each character is consumed once; the punctuator check is bounded by 27 candidates of length ≤ 2.

5.2 Sample output: --dump-tokens

```
$ node bin/tether.js build examples/fib.tth --dump-tokens
3:1+2    Keyword  fn            ← line 3, column 1, length 2
3:4+3    Ident    fib
3:7+1    Punct    (
3:8+1    Ident    n
3:9+1    Punct    :
3:11+3   Keyword  int
3:14+1   Punct    )
3:15+1   Punct    :
3:17+3   Keyword  int
3:21+1   Punct    {
4:3+2    Keyword  if
...                            (127 tokens in total, ending in EOF)
```

5.3 Lexical errors

The lexer never stops at the first problem. It records a diagnostic, produces a token that is marked as already reported, and keeps scanning.

Error	Where the caret goes	Why
Unterminated string literal	On the **opening quote** (FR-L4)	The end of the line is not where the mistake is; the opening quote identifies which string is broken
Unknown escape, e.g. `\q`	On the two-character escape	Points at exactly the bad sequence inside the string
Unexpected character	On the character; adds "did you mean '&&'?" for & and || for |	Common mistake for C/Java programmers

Error	Where the caret goes	Why
Integer literal too large	Under the whole number	Above 2^53 − 1, JavaScript numbers cannot represent integers exactly, so the program would silently compute wrong answers
Control character	Shown as `U+0007` etc.	Invisible characters must still be identifiable

```
$ node bin/tether.js build test/fixtures/lex-errors.tth
test/fixtures/lex-errors.tth:1:9: error: unterminated string literal
  1 | let s = "never closed;
    |         ^
test/fixtures/lex-errors.tth:2:14: error: unknown escape sequence '\q'
  2 | let t = "bad \q escape";
    |              ^^
test/fixtures/lex-errors.tth:3:11: error: unexpected character '&'; did you mean '&&'?
  3 | let u = 1 & 2;
    |           ^
test/fixtures/lex-errors.tth:4:9: error: integer literal is too large
  4 | let v = 99999999999999999999;
    |         ^^^^^^^^^^^^^^^^^^^^
test/fixtures/lex-errors.tth:5:10: error: unexpected character '#'
  5 |   let w = #;          (line starts with a TAB; the caret line reuses it)
    |           ^
5 errors; no output written
```

Cascade suppression in action
The parser runs over this file too, but reports **zero** extra errors. Every statement it trips on involves a token the lexer already reported, so the parser treats those failures as cascades and stays quiet (section 7.4).

6. The frozen grammar

The grammar was frozen at the start of implementation (PRD section 4) and is recorded in `docs/grammar.md`; it becomes Appendix A of the final report. Any change requires a revision of the PRD, which protects against scope creep (PRD risk table).

```
program      ::= { fnDecl | statement } EOF
fnDecl       ::= "fn" IDENT "(" [ param { "," param } ] ")" [ ":" type ] block
param        ::= IDENT ":" type
type         ::= "int" | "bool" | "string"
block        ::= "{" { statement } "}"

statement    ::= letDecl ";" | ifStmt | whileStmt | forStmt | returnStmt
               | printStmt | block | simpleStmt ";"
letDecl      ::= "let" IDENT [ ":" type ] "=" expr
simpleStmt   ::= IDENT assignOp expr | expr
assignOp     ::= "=" | "+=" | "-=" | "*=" | "/=" | "%="
ifStmt       ::= "if" "(" expr ")" block [ "else" ( ifStmt | block ) ]
whileStmt    ::= "while" "(" expr ")" block
forStmt      ::= "for" "(" [ letDecl | simpleStmt ] ";" [ expr ] ";" [ simpleStmt ] ")" block
returnStmt   ::= "return" [ expr ] ";"
printStmt    ::= "print" "(" expr ")" ";"

expr         ::= orExpr
orExpr       ::= andExpr { "||" andExpr }
andExpr      ::= eqExpr  { "&&" eqExpr }
eqExpr       ::= relExpr { ( "==" | "!=" ) relExpr }
relExpr      ::= addExpr { ( "<" | "<=" | ">" | ">=" ) addExpr }
addExpr      ::= mulExpr { ( "+" | "-" ) mulExpr }
mulExpr      ::= unary   { ( "*" | "/" | "%" ) unary }
unary        ::= ( "-" | "!" ) unary | primary
primary      ::= INT | STRING | "true" | "false"
               | IDENT [ "(" [ expr { "," expr } ] ")" ] | "(" expr ")"
```

Precedence	Operators	Associativity
1 (loosest)	`||`	left
2	&&	left
3	== !=	left
4	< <= > >=	left
5	+ −	left
6	* / %	left
unary (tightest)	prefix − !	right (prefix)

6.1 Properties of the grammar

- **No left recursion.** Repetition is written with `{ }` rather than `expr ::= expr "+" term`, which would make a recursive-descent parser loop forever.
- **No dangling else.** The branches of `if` must be blocks, so `else` always belongs to the nearest `if` unambiguously.
- **Almost LL(1).** Every choice can be made by looking at one token, except `simpleStmt`, where both `IDENT assignOp expr` and `expr` start with an identifier. The parser peeks at the second token to decide (LL(2) at that one point). The rule could be left-factored instead; peeking is simpler and gives clearer error messages.
- **Context rules enforced in the parser:** `fn` only at top level; assignment is a statement, not an expression (`if (x = 1)` is a syntax error); only named functions can be called.

7. Syntax analysis

`src/parser.js` is a hand-written **recursive-descent** parser (FR-P1). Each grammar rule has one method (`parseFnDecl`, `parseIf`, `parseFor`, ...) whose comment quotes the rule it implements. Binary operators are parsed by **precedence climbing** over a table, which replaces the six nested `orExpr→mulExpr` functions of the textbook form.

7.1 Precedence climbing

```
// Parses a binary expression whose operators all bind at least as tightly as minPrec.
parseExpr(minPrec = 1) {
  let left = this.parseUnary();
  for (;;) {
    const t = this.cur;
    const prec = t.kind === 'Punct' ? BINARY_PRECEDENCE.get(t.lexeme) : undefined;
    if (prec === undefined || prec < minPrec) return left;
    this.advance();
    const right = this.parseExpr(prec + 1);   // +1 makes the operator left-associative
    left = { kind: 'Binary', span: union(left.span, right.span), op: t.lexeme, left, right };
  }
}
```

Worked trace for `1 + 2 * 3 - 4`:

```
parseExpr(1): left = 1
  sees '+' (prec 5 ≥ 1) → right = parseExpr(6):
      left = 2
      sees '*' (prec 6 ≥ 6) → right = parseExpr(7): left = 3; sees '-' (5 < 7) → return 3
      left = (2 * 3); sees '-' (5 < 6) → return (2 * 3)
  left = (1 + (2 * 3))
  sees '-' (prec 5 ≥ 1) → right = parseExpr(6): left = 4; EOF → return 4
  left = ((1 + (2 * 3)) - 4)          ✓ correct precedence and left associativity
```

7.2 The abstract syntax tree

Nodes are plain objects `{ kind, span, ...fields }`. There are 21 node kinds. Punctuation and the helper non-terminals (`orExpr`, `andExpr`, ...) do not appear in the tree, which is what makes it an *abstract* syntax tree rather than a parse tree. `Paren` is kept on purpose so the span of the parentheses is not lost.

Group	Node kinds
Program structure	`Program`, `FnDecl`, `Param`, `TypeRef`, `Block`
Statements	`Let`, `Assign` (with op: =, +=, ...), `If`, `While`, `For`, `Return`, `Print`, `ExprStmt`
Expressions	`Binary`, `Unary`, `Call`, `Paren`
Leaves	`Ident`, `IntLit`, `StringLit`, `BoolLit`

```
$ node bin/tether.js build examples/fib.tth --dump-ast
Program @3:1+318
  FnDecl @3:1+90
    name: Ident fib @3:4+3
    Param @3:8+6
      name: Ident n @3:8+1
      type: TypeRef int @3:11+3
    returnType: TypeRef int @3:17+3
    body: Block @3:21+70
      If @4:3+30
        cond: Binary < @4:7+5
          left: Ident n @4:7+1
          right: IntLit 2 @4:11+1
        then: Block @4:14+19
          Return @5:5+9
            value: Ident n @5:12+1
      Return @7:3+31
        value: Binary + @7:10+23
          left: Call @7:10+10
            callee: Ident fib @7:10+3
            ...                                    (83 nodes for fib.tth)
```

7.3 Error recovery: panic mode at statement granularity (FR-P3)

When the parser meets an unexpected token it records an error and throws a private `ParseError`. That exception unwinds to the nearest **statement loop** (the top level or the enclosing block), which calls `synchronize()` and carries on with the next statement.

```
recovering(parseFn) {                    // wraps every statement the loops parse
  const start = this.pos;
  try { return parseFn(); }
  catch (e) {
    if (!(e instanceof ParseError)) throw e;
    this.synchronize();                  // skip to a statement boundary
    if (this.pos === start) this.advance();   // always make progress: no infinite loops
    return null;                         // the broken statement is dropped from the AST
  }
}

synchronize() {   // stop at: EOF, '}', a statement keyword (fn let if while for return print '{'),
                  // or just after a ';'
}
```

- **Errors inside a function body recover inside that body.** The block's own statement loop catches the error, so the rest of the function is still parsed.
- **A missing ; is reported just after the previous token**, using a zero-length `spanAfter()` span, instead of on the next token, which may be several lines away.

- **A nested `fn` is parsed, then rejected** with one clear error. Rejecting it immediately would desynchronise the parser inside the body and cause a cascade.
- **Limit of 20 syntax errors**, then a note "too many syntax errors (20); stopping" (FR-P3).

7.4 Avoiding cascading errors

A cascade is one real mistake producing many follow-on errors. Two mechanisms prevent it. First, panic mode discards the rest of a broken statement instead of trying to make sense of it. Second, tokens the lexer already reported carry `reported: true`; if the parser fails at or just after such a token, it unwinds **without** reporting, because the programmer has already been told about that spot.

```
$ node bin/tether.js build test/fixtures/syntax-errors.tth      # seven independent mistakes
syntax-errors.tth:4:18: error: expected ':' after parameter 'b', found keyword 'int'
syntax-errors.tth:8:13: error: expected an expression, found ';'
syntax-errors.tth:9:9: error: expected ';' after print statement, found keyword 'let'
syntax-errors.tth:10:15: error: expected ')' to close the parenthesised expression, found ';'
syntax-errors.tth:11:4: error: expected '(' after 'if', found identifier 'x'
syntax-errors.tth:14:14: error: expected ')' after while condition, found '{'
syntax-errors.tth:17:5: error: expected variable name after 'let', found '='
7 errors; no output written                                          (exit code 1)
          (each error is also followed by its source line and caret line; omitted here for space)
```

8. Diagnostics and command-line interface

8.1 Caret diagnostics (FR-S3)

```
examples/bad.tth:3:9: error: unterminated string literal
  3 | let s = "hello;
    |         ^
```

- Header `file:line:col: severity: message`, then the offending source line with a line-number gutter, then a run of ^ exactly as wide as the span.
- The space before the carets **copies tab characters** from the source line, so carets stay aligned whatever tab width the terminal uses (tested).
- A zero-length span (a missing token) still gets one caret. A span crossing lines is underlined to the end of its first line.
- Diagnostics are **sorted by source offset** before printing (the lexer finishes before the parser starts, so without sorting all lexical errors would come first). Notes without a span come last.
- This renderer will be reused unchanged for semantic errors in milestone 3.

8.2 Command line (FR-C1 – FR-C3)

```
tether build <file> [-o <dir>] [-O1] [--no-map]
                    [--dump-tokens] [--dump-ast] [--dump-core] [--dump-mappings]
```

Flag	Status today	Behaviour
`--dump-tokens`	Done	Prints one token per line: `line:col+length kind lexeme`
`--dump-ast`	Done	Prints the indented tree with field names and spans
`-o <dir>`, `-O1`, `--no-map`	Partial	Parsed and validated; they take effect once code generation exists
`--dump-core`	Planned	Exits 2: "not implemented yet; it arrives with milestone 4 (lowering)"
`--dump-mappings`	Planned	Exits 2: "not implemented yet; it arrives with milestone 5 (mapping table)"

Exit code	Meaning (FR-C2)
0	Success. Today: the file lexes and parses cleanly (a note says no `.js` is written yet)
1	Compile error: at least one error diagnostic, or the input file cannot be read. No output is written (FR-S4)
2	Internal compiler error (an exception, printed with its stack) or a usage error such as an unknown flag

Flags for later milestones fail loudly instead of silently doing nothing; a test checks this. That way nobody can mistake a missing feature for a working one during a demo.

9. The playground web page

To demonstrate the front end visually, a single-page **playground** was built (`playground/index.html`). Crucially it is not a mock-up: it imports the compiler's own modules (`src/lexer.js`, `src/parser.js`, `src/diagnostics.js`, ...) directly, so what it shows is exactly what the command-line compiler computes.

- **Phase strip** across the top: green for phases that exist and succeeded (with token and node counts), red with an error count when a phase fails, grey with the milestone number for phases not built yet.
- **Live compilation.** Edit the source and the tokens, tree and diagnostics update as you type (debounced by 120 ms). Errors are underlined in the editor with a wavy red line.
- **Span highlighting.** Hover any token, tree node or error to highlight its exact span in the source; click it to select that text.
- **Cursor tracking.** Moving the cursor in the source selects the matching token or the deepest tree node that contains it.
- **Position readout.** The status bar shows each span as `line:col+length` and, in blue, the 0-based position a source map will store, computed by the real `toMapPosition()`.
- **Deep links** such as `#fib` or `#syntax-errors` open a program directly, which is handy in a demo. Light and dark themes follow the system setting. Works fully offline.

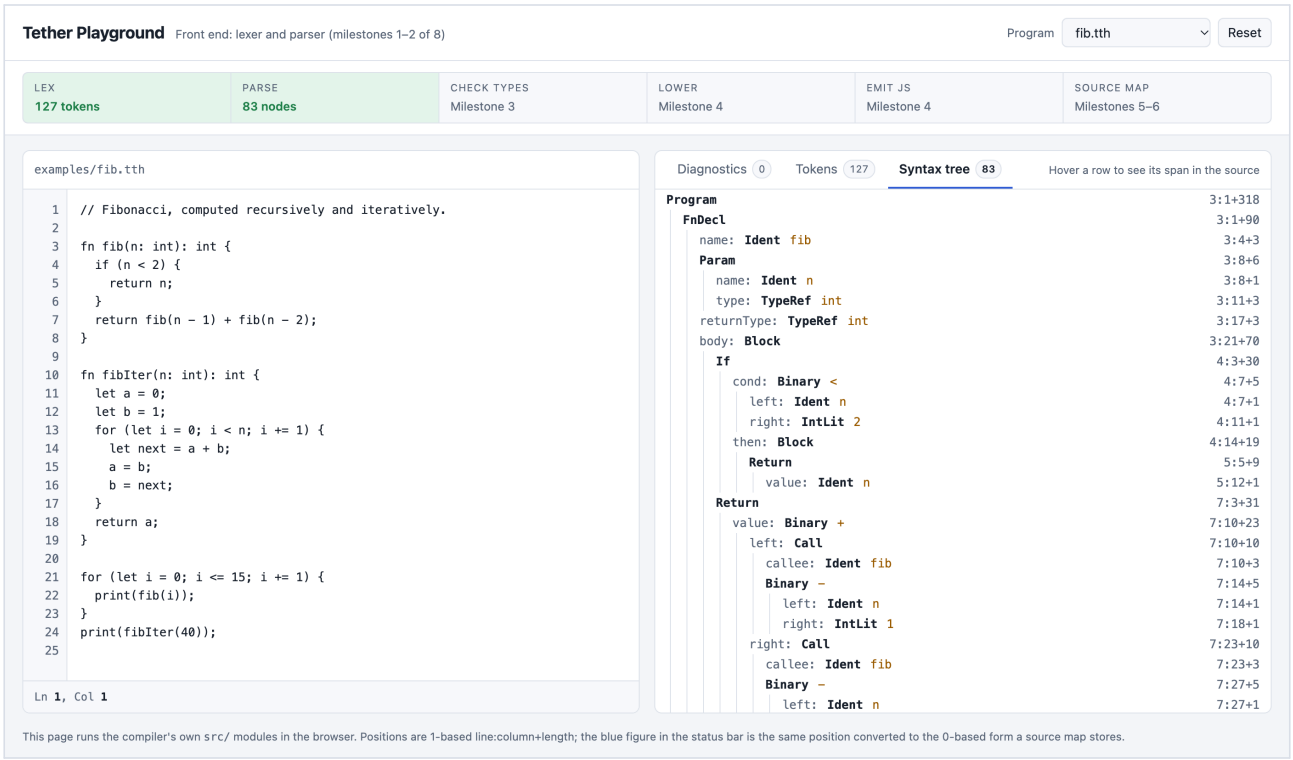


Figure 2. fib.tth: 127 tokens, 83 tree nodes, no errors. The syntax tree tab shows every node with its span.

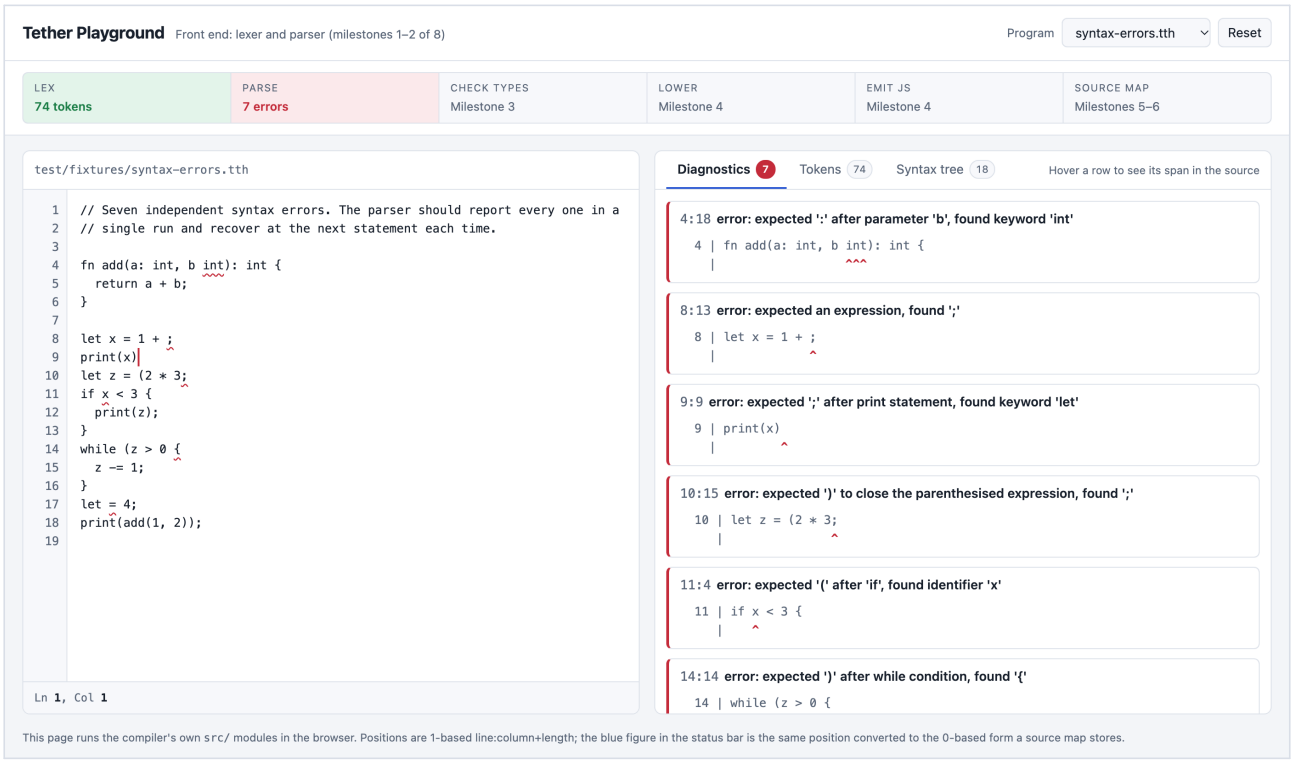


Figure 3. syntax-errors.tth: seven independent syntax errors reported in one run, each underlined in the source and shown with carets.

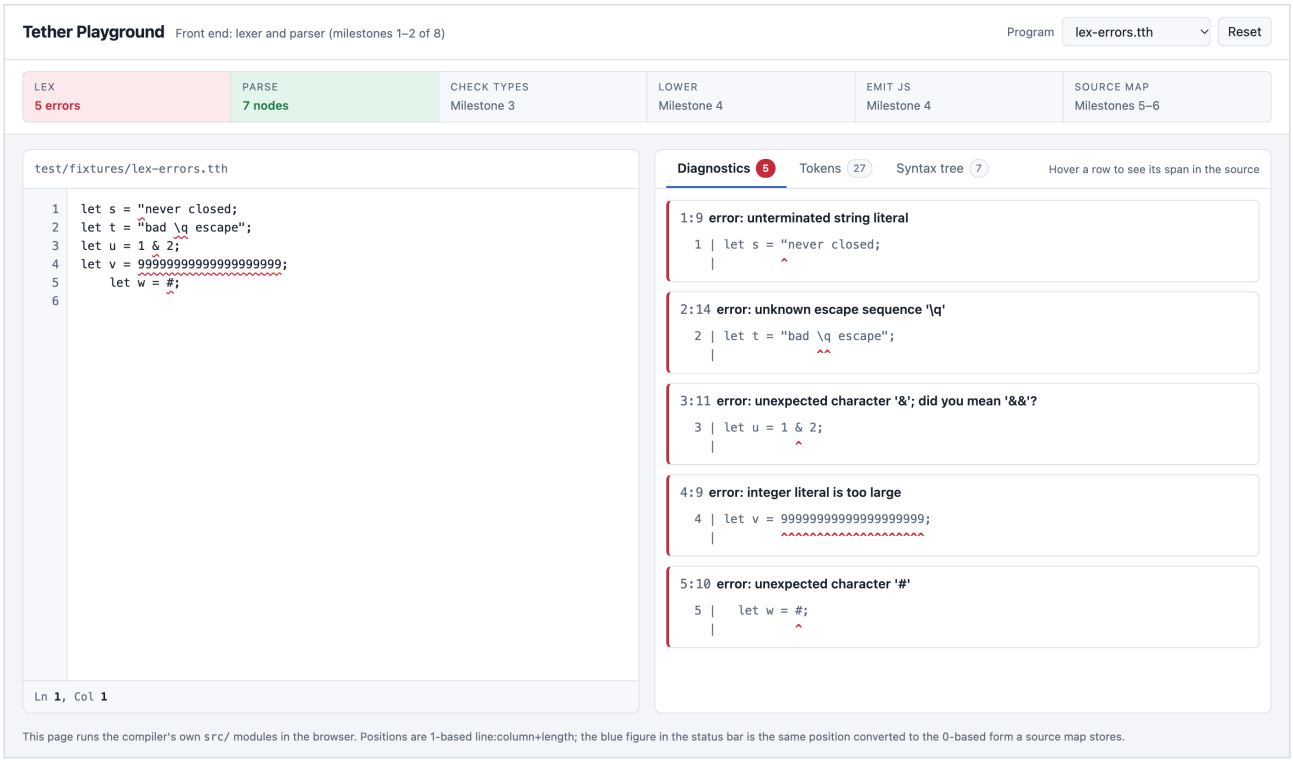


Figure 4. lex-errors.tth: five lexical errors. The Parse phase reports no extra errors because of cascade suppression.

9.1 How it is served

Browsers refuse to load ES modules from `file://` pages, so `npm run playground` starts a tiny static server (`scripts/serve.js`, standard library only). It listens on `127.0.0.1` only, rejects any path that resolves outside the project folder (403), and disables caching so edits to `src/` show up on reload. A test follows every import the playground makes and fails if any module reaches a `node:*` built-in, which guarantees the compiler core stays browser-safe.

10. Testing, verification and requirements status

10.1 Test suite

`npm test` runs 63 tests with Node's built-in runner. All pass.

File	Tests	What it covers
`test/span.test.js`	6	`union` (order independence, containment, null identity), `spanAfter`, `formatSpan`, and a **property test**: for random text containing tabs, CRLF and astral characters, `toMapPosition` always returns the independently computed 0-based position
`test/lexer.test.js`	13	Spans on every token; all literal, keyword and operator kinds; longest match; comments; tab = 1 column; three kinds of line end; UTF-16 columns; escapes; each lexical error and its caret position; EOF span
`test/parser.test.js`	18	Precedence table; left associativity; unary binding; calls and parentheses; every statement form; statement spans include keyword and `;`; **span invariant over all 4 benchmarks**; multi-error recovery; missing `;` placement; recovery inside function bodies; missing `}`; nested `fn`; no infinite loop on stray tokens; 20-error cap; no lexer→parser cascades
`test/diagnostics.test.js`	5	Header, source line and carets; tab reuse; zero-length span; multi-line span; ordering
`test/cli.test.js`	19	Golden files for `--dump-tokens` and `--dump-ast` on all 4 benchmarks; **determinism** (NFR-2); golden diagnostics for both error fixtures with exit code 1; exactly seven syntax errors; exit codes; full flag set accepted; later-milestone flags say so
`test/playground.test.js`	2	Playground imports only browser-safe modules; every example it offers exists

10.2 Golden-file testing

The expected output of each dump and each error fixture is stored in `test/golden/` and compared byte for byte. After a deliberate format change, `npm run test:update-golden` regenerates the files and the change is reviewed as a diff. This catches any unintended change in token positions, tree shape or message wording.

10.3 Measured performance

Benchmark	Source lines	Tokens	AST nodes	Lex + parse time
`fib.tth`	24	127	83	0.04 ms
`primes.tth`	25	108	67	0.08 ms
`collatz.tth`	28	132	85	0.07 ms
`fizzbuzz.tth`	32	169	104	0.08 ms

Times are averaged over 100 runs after a warm-up, on Node 25 on an Apple-silicon Mac. NFR-3 allows 200 ms for the *whole* pipeline, so the front end uses well under 0.1% of the budget. This will be re-measured once all phases exist.

10.4 Requirements traceability

ID	Requirement (short)	Status	Evidence / plan
FR-L1	Every token carries file, line, column, length	Done	Lexer test; `--dump-tokens` goldens
FR-L2	1-based in diagnostics, 0-based in map, one conversion	Done	`toMapPosition()` + property test; map writer will use it (M6)
FR-L3	Literals, identifiers, keywords, operators, comments	Done	Lexer tests
FR-L4	Unterminated string reported at opening quote	Done	Lexer test; lex-errors golden
FR-L5	Tab = 1 column, documented	Done	`docs/positions.md`; lexer test
FR-P1	Recursive descent with precedence climbing	Done	`parseExpr(minPrec)`; precedence tests
FR-P2	Node span = union of children; leaf = token	Done	Span-invariant test over every benchmark node
FR-P3	Recover at statement boundary, up to 20 errors	Done	7-error fixture; 20-error cap test
FR-P4	`--dump-ast` with spans	Done	Golden `*.ast` files
FR-S1	Scoped symbol table; shadowing; redeclaration error	Next	Milestone 3
FR-S2	Static type checking incl. missing return	Next	Milestone 3
FR-S3	file:line:col, source line, caret run	Done	Diagnostics tests; reused for semantic errors
FR-S4	No codegen after errors; exit 1	Partial	Exit 1 and "no output written" today; codegen gate when emitter exists
FR-M1–M3	Lowering (for, compound assignment, &&/||), span inheritance, `--dump-core`	Planned	Milestone 4
FR-G1–G4	ES2015 emitter, single write path, reserved-word renaming, sourceMappingURL	Planned	Milestones 4 and 6
FR-X1–X5	Source Map v3 JSON, VLQ, delta encoding, coverage, sourcesContent	Planned	Milestones 5–6
FR-C1	CLI accepts build, -o, -O1, --no-map	Partial	All accepted and validated; take effect with codegen
FR-C2	Exit codes 0 / 1 / 2	Done	CLI test
FR-C3	Dump flags for each phase	Partial	Tokens and AST done; core and mappings exit 2 with a message
FR-O1–O3	–01 folding and DCE; `--map-report`	Planned	Milestone 8 (Could)
NFR-1	No parser generator or framework	Holds	Zero dependencies
NFR-2	Deterministic output	Holds	Determinism test
NFR-3	Compile under 200 ms	Holds	Front end < 0.1 ms per benchmark
NFR-4	Maps checked by an independent decoder	Planned	Milestone 6
NFR-5	No runtime library in output	Planned	By design of the emitter

ID	Requirement (short)	Status	Evidence / plan
NFR-6	Phases unit-testable; golden end-to-end tests	**Holds**	One test file per phase; CLI goldens

Acceptance criteria
None of the acceptance criteria AC-1 to AC-6 is due yet: the first, AC-4 (five semantic errors in one run), is the exit criterion of milestone 3. Its syntax-level counterpart, seven syntax errors in one run with correct carets, is already demonstrated by `syntax-errors.tth`.

10.5 Known limitations at this stage

- No semantic checks yet: `print(undefinedName + true);` lexes and parses without error today. Milestone 3 will reject it (undeclared name; `int + bool` mismatch).
- No JavaScript output yet, so nothing can be run; the CLI says so explicitly.
- Recovery is per statement: two mistakes inside the same statement report only the first.
- Identifiers are ASCII only (a frozen-grammar decision); non-ASCII text is allowed in strings and comments.

11. Plan for the remaining 75%

This section explains how each later milestone will be built, so it can be discussed in the viva. Everything here is **planned design**, not yet implemented. Designs may be refined during implementation; any change to the language itself requires a PRD revision.

11.1 Milestone 3: semantic analysis

- **Symbol table** as a stack of scopes, each a map from name to symbol `{ kind, type, declSpan }`. A scope is pushed for each function and block. Lookup searches innermost to outermost. Declaring a name already in the *same* scope is an error, with a note pointing at the first declaration; shadowing in a nested block is legal (FR-S1).
- **Two passes over the program.** Pass 1 records every top-level function signature, so functions can be called before they are declared and can be mutually recursive. Pass 2 checks bodies and top-level statements in order.
- **Type checking** (FR-S2) annotates each expression node with its type; the AST spans are not changed.
- **Definite-return analysis** for functions with a return type: a `return` returns; a block returns if any statement in it returns; an `if` returns only if it has an `else` and both branches return; loops are treated conservatively as possibly not running.
- **No cascades.** An erroneous expression gets a special *error type* that is compatible with everything, so `undeclared + 1` produces one error, not two. This is what makes AC-4 (five errors in one run) achievable.

Construct	Rule (planned)
`+ - * / %`	`int`, `int` → `int` (whether `string + string` means concatenation is decided and documented at the start of M3)
`< <= > >=`	`int`, `int` → `bool`
`== !=`	both operands of the same type → `bool`
`&& ||`, `!`	`bool` operands → `bool`
unary `-`	`int` → `int`
`if` / `while` / `for` condition	must be `bool` (so `if (1)` is rejected)
`let x: T = e`	type of `e` must be `T`; without `: T` the type of `x` is inferred from `e`

Construct	Rule (planned)
Call `f(a, b)`	`f` must be a function; argument count and each argument type must match the parameters
`return e`	type of `e` must match the declared return type; bare `return` only in a function with no return type
Compound `x += e`	same rule as `x = x + e`

11.2 Milestone 4: lowering and a naive JavaScript emitter

Lowering rewrites the checked AST into a smaller **core** language so the emitter has fewer cases (FR-M1). Every node created by lowering inherits the span of the construct that caused it (FR-M2), so stepping through synthesized code still maps back to the right source line. `--dump-core` prints the result (FR-M3).

```
Source construct                          Lowered core form
for (let i = 0; i < n; i += 1) { B }  →  { let i = 0; while (i < n) { B  i = i + 1; } }
x += e                                →  x = x + e                (span of the whole "x += e")
a && b                                →  a ? b : false
a || b                                →  a ? true : b
```

- Moving the `for` update to the end of the body is safe because Tether has no `break` or `continue`. The extra block keeps the loop variable scoped to the loop.
- **Emitter** owns the only write path to the output buffer and updates output line and column on every write (FR-G2). An assertion after each compile recomputes the counters from the final string.
- **Mapping** of Tether to JS: `fn` → `function`, `let` → `let`, `print(e)` → `console.log(e)`, `==` → `===` (operands have the same static type), integer `/` → `Math.trunc(a / b)` because JavaScript division is floating point. `%` maps directly (both truncate toward zero).
- **Reserved words** (FR-G3): a Tether variable named like a JavaScript reserved word (fizzbuzz uses `var`) is renamed deterministically, e.g. `var$`. Since `$` cannot appear in a Tether identifier, the new name can never clash with a user name. The original name goes into the map's `names` array.
- **Shadowing hazard found during design:** in Tether, `{ let x = x + 1; }` inside a block where an outer `x` exists reads the outer `x`. The same code in JavaScript throws a ReferenceError (temporal dead zone). The emitter will therefore rename a shadowing declaration whose initializer mentions the outer name (e.g. `x$1`).
- Exit criterion AC-1: all four benchmarks print exactly their expected output under Node.

11.3 Milestones 5–6: Base64 VLQ and the source map

The map file is JSON with `version: 3`, `file`, `sources`, `sourcesContent` (the embedded original text, FR-X5), `names` and `mappings` (FR-X1). The last line of the `.js` file is `//# sourceMappingURL=fib.js.map` (FR-G4).

The mappings string (FR-X2): one group per generated line separated by `;`, segments within a line separated by `,`. Each segment has 4 fields (generated column, source index, source line, source column), or 5 when a name applies (plus name index). All values are 0-based and **delta-encoded** (FR-X3): the generated column restarts at each new generated line; the other four fields accumulate across the whole file.

Base64 VLQ: move the sign into the lowest bit (`n ≥ 0 → 2n`, `n < 0 → 2|n| + 1`), split into 5-bit groups starting from the least significant, set bit 6 (value 32) on every group except the last, and write each 6-bit value as a Base64 character.

Value	After sign bit	Groups	VLQ
0	0	0	A
1	2	2	C

Value	After sign bit	Groups	VLQ
−1	3	3	D
−5	11	11	L
15	30	30	e
16	32	0 + continuation (32), then 1	gB

Hand-computed example of what the map will contain for the first two generated lines of `fib.js` (names array `["fib", "n"]`):

```
Generated JS                  Segment (absolute values)              Deltas          VLQ
line 0: function fib(n) {     col 0  → src line 2, col 0             [0,0,2,0]       AAEA
                              col 9  → src line 2, col 3, name "fib" [9,0,0,3,0]     SAAGA
                              col 13 → src line 2, col 7, name "n"   [4,0,0,4,1]     IAAIC
line 1:   if (n < 2) {        col 2  → src line 3, col 2             [2,0,1,-5]      EACL
                              col 6  → src line 3, col 6, name "n"   [4,0,0,4,0]     IAAIA

mappings: "AAEA,SAAGA,IAAIC;EACL,IAAIA;..."
(Tether line 3 "fn fib(n: int)" is 0-based line 2; the generated column restarts on line 1,
 but the source column carries over from 7, giving the negative delta -5.)
```

- **Milestone 5** tests the VLQ encoder against published test vectors **before** any integration (PRD risk mitigation for wrong delta encoding).
- **Milestone 6** validates every map with an **independent reference decoder** (NFR-4): decoding must succeed; every segment must point at a real position in the source; every name index must be in range; and for every entry in the compiler's own mapping table, querying the decoded map must return the same original position in 100% of cases (AC-3).
- At minimum, a mapping is emitted for the first token of every statement and for every identifier reference (FR-X4).

11.4 Milestones 7–8: browser proof and optimisation

- **Milestone 7:** load the generated JS in Chrome DevTools; set breakpoints on at least ten distinct Tether statements and show each one halts at the right statement (AC-2); throw a runtime exception and show the stack trace names the `.tth` file and line (AC-5). Screenshots and a written walkthrough.
- **Milestone 8 (Could priority):** `-O1` constant folding (`2 * 3` → `6`) and dead-code elimination (`if (false) {...}`, code after `return`). A folded expression takes the span of the whole expression it replaces (FR-O2). `--map-report` prints the share of source statements that still have a mapping at `-O0` and `-O1` across all four benchmarks (AC-6).

Week	Deliverable	Exit check
3	Scopes, symbol table, type checker, definite-return analysis, semantic-error fixture	AC-4: five semantic errors in one run, correct carets
4	Lowering pass, `--dump-core`, naive emitter, reserved-word renaming	AC-1: four benchmarks give exact expected output
5	VLQ encoder, mapping table, `--dump-mappings`	Encoder passes published vectors
6	`.js.map` writer, sourceMappingURL, reference-decoder tests	AC-3: 100% round trip
7	Chrome DevTools breakpoints and stack traces	AC-2 and AC-5 screenshots
8	`-O1`, `--map-report`, final report and presentation	AC-6 figures recorded

11.5 Risks and decisions log

Item	Status
Off-by-one between 1-based and 0-based (risk: High)	Mitigated now: single `toMapPosition()` with property test
Wrong delta encoding: map loads but points wrong (High)	Will be mitigated: VLQ test vectors in week 5, independent decoder from week 6
Emitter column drift (Medium)	Will be mitigated: single write path + end-of-compile assertion
Scope creep (Medium)	Mitigated now: grammar frozen in `docs/grammar.md`
Optimisation displacing core work (Medium)	−01 not started until AC-1 to AC-5 pass
Open question 3: tab width	**Decided:** 1 column, matching Source Map v3 and DevTools
Open question 1: expression- vs statement-level mappings	Open; minimum is FR-X4 (statements + identifiers)
Open question 2: need for `sourceRoot`	Open; a single relative source path is expected to suffice

12. Viva preparation: likely questions and answers

Questions are grouped by topic. Groups A–F cover work that is **done**; groups G–J cover work that is **planned**; group K covers general compiler-design theory as it applies to this project. Answers are written to be said aloud in under a minute.

A. The project and its scope

Q1. What is Tether, in one sentence?

A compiler for a small statically typed language that outputs ES2015 JavaScript plus a Source Map v3 file, so a normal JavaScript debugger shows errors, breakpoints and stack traces in the original `.tth` source.

Q2. Why do you call it a transpiler and not a compiler?

It is source-to-source: the target is another high-level language (JavaScript), not machine code. All the classic phases still exist (lexing, parsing, semantic analysis, an intermediate form, code generation); only the back-end target is different.

Q3. What exactly is finished at 25%?

Milestones 1 and 2 of 8: the frozen grammar, the lexer with spans, the recursive-descent parser building an AST with spans, panic-mode error recovery, caret diagnostics (pulled forward from milestone 3), the CLI with `--dump-tokens`, `--dump-ast` and exit codes, 63 passing tests, and a browser playground.

Q4. Why is source position so central to this project?

A source map is just a table from generated position to original position. If any phase loses or shifts a position, the map still loads but silently points to the wrong line. So spans are attached from the very first token and the invariants are tested, rather than added at the end.

Q5. Why JavaScript as the target?

It runs everywhere, and browsers already ship debuggers that understand source maps, so I can prove debugging works without writing a debugger. ES2015 also has block-scoped `let`, which matches Tether's scoping.

Q6. Why leave out arrays, objects and closures?

They would add runtime and semantic work without adding new compiler-design content; the project is about carrying positions through every phase. The PRD lists them as out of scope, and the grammar is frozen to stop scope creep.

Q7. Do you use any libraries?

None. Only Node 18+ standard library; tests use the built-in `node:test`. NFR-1 forbids parser generators and compiler frameworks, and I extended that to zero dependencies.

B. Lexical analysis

Q8. How does your lexer work? Did you use regular expressions or a DFA?

It is a hand-written single-pass scanner. It looks at the current character and dispatches to a branch: whitespace, comment, number, identifier, string, punctuator, or error. Each branch is effectively a small DFA written as a loop. No regex engine is used for tokenising.

Q9. How do you distinguish keywords from identifiers?

Scan the longest run of identifier characters, then look the word up in the `KEYWORDS` set. `true` and `false` become `Bool` tokens with a value. Because of maximal munch, `iffy` is one identifier, not the keyword `if` followed by `fy`.

Q10. What is maximal munch and where do you use it?

Always take the longest possible token. Identifiers and numbers consume every valid character, and the punctuator list is ordered longest first, so <= beats < and += beats +.

Q11. How is `a<=-b` tokenised?

`a` (Ident), <= (Punct), - (Punct), `b` (Ident). There is no =- operator, and the minus becomes a unary operator in the parser.

Q12. How do you keep line and column numbers correct?

A single `advance()` function is the only code that moves forward; it updates offset, line and column together, so they can never drift apart. `\n`, `\r\n` and a lone `\r` each end one line.

Q13. Why does a tab count as one column and not four?

Source Map v3 columns are plain UTF-16 indices and Chrome DevTools resolves them that way; expanding tabs would make every map column after a tab wrong. To keep carets aligned visually, the caret line copies the tab characters from the source line. This settles open question 3 of the PRD.

Q14. Why count columns in UTF-16 code units?

Because JavaScript strings and source-map consumers do. A character outside the Basic Multilingual Plane, like an emoji, counts as two columns. A lexer test checks this.

Q15. Which lexical errors do you detect?

Unterminated string (caret on the opening quote, FR-L4), unknown escape (caret on the two-character escape), unexpected character (with a "did you mean `&&`?" hint for `&` and `||` for `|`), integer literal too large, and control characters shown as `U+XXXX`.

Q16. Why reject integers above 2^53 − 1?

The target represents numbers as IEEE-754 doubles, which cannot store larger integers exactly. Accepting them would make the compiled program compute wrong answers silently; a compile-time error is better.

Q17. Does the lexer stop at the first error?

No. It records a diagnostic, emits a token marked `reported`, and keeps scanning, and the parser still runs afterwards. That is how one run can report many problems (goal G4).

Q18. Why is `-5` not a single integer token?

If the lexer handled the sign, `a-5` would need context to decide between subtraction and a negative literal. Treating - as a unary operator keeps the lexer context-free and the grammar simple.

Q19. What is the time complexity of the lexer?

Linear in the length of the source: each character is consumed once, and the punctuator check is bounded by 27 candidates of length at most 2.

C. Grammar and parsing

Q20. What kind of grammar is it? Is it LL(1)?

A context-free grammar in BNF with EBNF repetition. It is LL(1) everywhere except one point: in `simpleStmt`, both `IDENT assignOp expr` and `expr` start with an identifier, so the parser peeks one token further (LL(2) there). It could be left-factored, but peeking is simpler and gives better messages.

Q21. Why recursive descent instead of an LR parser generator like yacc or Bison?

NFR-1 rules out generators. Beyond that, recursive descent has one function per rule, so it is easy to read and debug, attaching spans is trivial, and I have full control over error messages and recovery.

Q22. Explain precedence climbing.

`parseExpr(minPrec)` parses a unary expression as the left side, then loops: while the next operator's precedence is at least `minPrec`, consume it, parse the right side with `parseExpr(prec + 1)`, and combine. Passing `prec + 1` makes operators left-associative; passing `prec` would make them right-associative. A table replaces six nested functions.

Q23. Trace how `1 + 2 * 3 - 4` is parsed.

Left is 1; + (prec 5) recurses with min 6, which parses 2, then * (prec 6) binds 3, then stops at – because 5 < 6, giving `2*3`. Back at the top, left is `1 + (2*3)`; then – (prec 5 ≥ 1) parses 4. Result: `((1 + (2 * 3)) - 4)`. See section 7.1.

Q24. How do you avoid left recursion?

The grammar uses repetition `{ ... }` instead of rules like `expr ::= expr "+" term`, which would make a recursive-descent parser recurse forever. Precedence climbing turns the repetition into a loop.

Q25. Is there a dangling-else problem?

No. The branches of `if` must be blocks in braces, so it is always clear which `if` an `else` belongs to. `else if` is handled by `parseIf` calling itself.

Q26. Why is assignment a statement, not an expression?

It prevents the classic `if (x = 1)` bug and keeps type checking simple. `if (x = 1)` and `x = y = 1` are both syntax errors with clear messages.

Q27. What is the difference between your AST and a parse tree?

A parse tree has a node for every grammar symbol, including punctuation and the helper levels like `orExpr` and `mulExpr`. The AST keeps only meaningful structure, for example a single `Binary +` node. The spans still cover the consumed punctuation, and `Paren` is kept so the span of the parentheses is not lost.

Q28. How many node kinds are there and how do you walk the tree?

21 kinds. `CHILD_FIELDS` in `ast.js` lists the child fields of each kind in source order; the dumper, the playground and the tests all walk through it, so adding a kind is a one-line change.

Q29. What is the span invariant and how is it tested?

A leaf takes its token's span; an interior node's span is the union of its children and the tokens it consumed directly. A test visits every node in all four benchmarks and checks children are inside the parent, in order, non-overlapping, and that every leaf matches a real token.

Q30. Why parse a nested `fn` fully if it is not allowed?

If the parser rejected it at the `fn` keyword, it would lose track inside the body and produce a cascade of bogus errors. Parsing it and then reporting one error, "functions can only be declared at the top level", keeps recovery clean.

Q31. What is the complexity of the parser?

Linear in the number of tokens. There is no backtracking; decisions use one token of lookahead (two at one point).

D. Positions and spans

Q32. What does a span contain, and why store both an offset and line/column?

`file`, `offset`, `length`, `line`, `col`, `endLine`, `endCol`. The offset and length make slicing and highlighting cheap; line and column are what humans and source maps use; the end position handles nodes that span several lines.

Q33. Where do 1-based positions become 0-based?

Only in `toMapPosition()` in `span.js`. Everything inside the compiler and every diagnostic is 1-based; source-map fields are 0-based. No other code is allowed to subtract 1 from a line or column, which removes a whole class of off-by-one bugs.

Q34. How do you test that conversion?

With a property test: it generates random text containing tabs, CRLF line ends and astral characters, picks random positions, and checks `toMapPosition` against a 0-based position computed independently.

Q35. What does `union()` do?

Returns the smallest span covering two spans, whichever order they are given, treating `null` as the identity. Interior node spans are built with it.

Q36. Why does a missing `;` point after the previous token?

The next token could be lines away. `spanAfter()` makes a zero-length span right after the previous token, which is where the `;` belongs, and the renderer still draws one caret there.

E. Error handling and diagnostics

Q37. What is panic-mode recovery and how did you implement it?

On a syntax error the parser throws `ParseError`; `recovering()` in the nearest statement loop catches it, `synchronize()` skips tokens until just after a `;` or before a statement keyword, `{`, `}` or end of file, and parsing continues with the next statement.

Q38. How do you guarantee recovery cannot loop forever?

If a statement failed without consuming anything, `recovering()` forces one token forward, so every iteration makes progress. A test feeds stray tokens to prove it terminates.

Q39. What are cascading errors and how do you avoid them?

Follow-on errors caused by one real mistake. Panic mode throws away the rest of the broken statement, and tokens the lexer already reported are flagged, so the parser stays quiet when it fails on them. The lex-errors fixture shows 5 lexical errors and 0 extra parser errors.

Q40. Why stop after 20 errors?

FR-P3. After that many, later errors are mostly noise, and the user is better off fixing the first ones. A note says the parser stopped.

Q41. Why recover per statement and not per expression?

It is simple and predictable and meets goal G4. Phrase-level recovery, such as inventing missing tokens, risks building misleading trees. The trade-off is that two mistakes in the same statement report only the first.

Q42. How are diagnostics ordered?

Sorted by source offset, with span-less notes last. Without sorting, all lexer errors would come before all parser errors, because the lexer finishes first.

Q43. What happens when a program has errors?

Every diagnostic is printed with its caret, then "N errors; no output written", and the process exits with 1 (FR-S4). An unexpected exception exits with 2 and prints "internal compiler error" with the stack; usage errors also exit 2.

F. Testing, tooling and the playground

Q44. How have you tested the implementation?

63 tests on `node:test`: unit tests per phase (span 6, lexer 13, parser 18, diagnostics 5), end-to-end CLI tests with golden files (19), and playground safety tests (2). All pass.

Q45. What are golden-file tests?

The expected output is stored in `test/golden/` and compared byte for byte. After a deliberate format change, `npm run test:update-golden` regenerates the files and the diff is reviewed. They catch any unintended change in positions, tree shape or messages.

Q46. How do you show the compiler is deterministic (NFR-2)?

A test compiles each benchmark twice and compares the outputs. The code avoids timestamps, randomness and anything that depends on hash ordering.

Q47. Is the playground just a mock-up?

No. It imports the compiler's actual `src/` modules, the same code the CLI runs. It re-lexes and re-parses as you type, highlights spans on hover, and selects the matching node when you move the cursor.

Q48. Why does the playground need a server?

Browsers block ES modules on `file://` pages. `scripts/serve.js` serves the project folder on 127.0.0.1 only, rejects paths outside the folder, and disables caching.

Q49. How do you ensure the compiler keeps working in the browser?

A test follows the playground's imports transitively and fails if any module imports a `node:*` built-in. Only `cli.js` uses Node APIs, and the playground does not load it.

Q50. How fast is it?

Lexing and parsing each benchmark takes under 0.1 ms on average. NFR-3 allows 200 ms for the whole pipeline; I will re-measure once all phases exist.

G. Semantic analysis (milestone 3, planned)

Q51. How will the symbol table work?

A stack of scopes, each a map from name to `{ kind, type, declSpan }`. Push a scope for each function and block, look names up from innermost to outermost. Redeclaring in the same scope is an error with a note at the first declaration; shadowing in a nested block is allowed (FR-S1).

Q52. How will you handle calls to functions declared later, or mutual recursion?

Two passes: first collect every top-level function signature into the global scope, then check bodies. JavaScript function declarations are hoisted, so the generated code works the same way.

Q53. What type errors will you detect?

Mismatched operand types, non-boolean conditions, wrong argument count, wrong argument type, a `return` value of the wrong type, and a missing return on a value-returning path (FR-S2). For example, `print(undefinedName + true);` passes today but will give an undeclared-name error.

Q54. How will you check for a missing return?

Definite-return analysis: `return` returns; a block returns if one of its statements returns; `if` returns only when it has an `else` and both branches return; loops are treated as possibly not running. If a function with a return type does not definitely return, report it.

Q55. How will you report five semantic errors in one run without cascades (AC-4)?

The checker never stops at the first error. An erroneous expression gets an "error type" that is compatible with everything, so `undeclared + 1` produces only the undeclared-name error, not a second type error.

Q56. What happens with `let x = x + 1;` in an inner block when an outer `x` exists?

In Tether, the initializer is checked before the new `x` is declared, so it reads the outer `x`. JavaScript would throw a ReferenceError for the same code because of the temporal dead zone, so the emitter must rename the inner `x` (for example `x$1`). I found this while designing the emitter.

Q57. Does semantic analysis change the tree?

It annotates nodes with resolved symbols and types (a decorated AST) but never changes spans.

H. Lowering and code generation (milestone 4, planned)

Q58. Why have a lowering pass at all, since JavaScript already has `for` and `&&`?

It shrinks the language the emitter has to handle to a small core, and it is where span inheritance (FR-M2) is tested most: synthesized nodes must still map back to the source. `for` becomes `while`, `x += e` becomes `x = x + e`, `a && b` becomes `a ? b : false`, and `a || b` becomes `a ? true : b`.

Q59. Doesn't turning `for` into `while` break `continue`?

Tether has no `continue` or `break`, so putting the update at the end of the body is safe. The loop is wrapped in a block so the loop variable stays scoped to it.

Q60. What span does the synthesized `x + e` get when lowering `x += e`?

The span of the whole `x += e` statement that caused it (FR-M2), so breakpoints and stepping still land on the original line.

Q61. How will the emitter track output positions?

It owns the only function that writes to the output buffer, and every write updates the output line and column (FR-G2). When it writes the first token of a node, it records the generated position against that node's span. An assertion recomputes the counters from the final string after each compile.

Q62. A variable is called `var` in fizzbuzz. What happens?

Names that are JavaScript reserved words are renamed deterministically, for example to `var$`. `$` cannot appear in a Tether identifier, so this can never clash with a user name. The original name is kept in the map's `names` array so the debugger can show `var` (FR-G3).

Q63. How will `print`, `==` and `/` be translated?

`print(e)` becomes `console.log(e)`. `==` becomes `===`, which is safe because both operands have the same static type. Integer `/` becomes `Math.trunc(a / b)` because JavaScript division is floating point; `%` maps directly because both truncate toward zero.

Q64. Why not use a code generator such as Babel's?

NFR-1, and because I need exact control of output columns for the map. A pretty-printer library would hide where each token ends up.

I. Source maps (milestones 5–7, planned)

Q65. What is a source map and how does the debugger find it?

A JSON file mapping positions in the generated file back to positions in the original source. The last line of the `.js` file is `//# sourceMappingURL=fib.js.map` (FR-G4), and the debugger loads the map from there, relative to the `.js` file's own directory.

Q66. Which fields will your map have?

`version` (3), `file`, `sources`, `sourcesContent`, `names` and `mappings` (FR-X1). `sourcesContent` embeds the original source so the debugger does not need access to the `.tth` file (FR-X5).

Q67. Explain the structure of the mappings string.

Generated lines are separated by `;` and segments within a line by `,`. Each segment has 4 fields (generated column, source index, original line, original column) or 5 with a name index. Every field is a Base64 VLQ number (FR-X2).

Q68. What is delta encoding, and what is the classic mistake?

Each field is stored relative to its previous value. The generated column resets at every new generated line, but the other four fields accumulate across the whole file (FR-X3). Resetting the source line or column per line is the classic bug: the map still loads but points to the wrong places. The example in section 11.3 shows a negative source-column delta of −5 for exactly this reason.

Q69. What is Base64 VLQ? Encode 16.

Variable-length quantity: shift the value left by one and put the sign in bit 0, split into 5-bit groups from the least significant end, set the continuation bit (32) on all but the last group, and write each 6-bit value as a Base64 character. 16 becomes 32 = groups 0 and 1, so the first is 0 + 32 = `g` and the second is 1 = `B`, giving `gB`. Also 0 = `A`, 1 = `C`, −1 = `D`.

Q70. How will you verify the map is correct (NFR-4, AC-3)?

Decode it with an independent reference decoder, not my own code, because a self-consistent encoder can still be wrong. For every entry in the compiler's mapping table, query the decoded map and require the same original position in 100% of cases. Also check that every segment points at a real source position and every name index is in range.

Q71. Statement-level or expression-level mappings?

The minimum (FR-X4) is the first token of every statement plus every identifier reference. Statement level is enough for breakpoints; expression level improves stack-trace columns. This is open question 1 in the PRD.

Q72. How will you demonstrate it working?

Milestone 7: in Chrome DevTools, set breakpoints on at least ten Tether statements and show each one halts at the right place (AC-2), and trigger a runtime error whose stack trace names the `.tth` file and line (AC-5), with screenshots.

J. Optimisation (milestone 8, planned)

Q73. What will `-O1` do?

Constant folding (`2 * 3` becomes `6`) and dead-code elimination (`if (false) { ... }`, statements after `return`). It is Could priority and will not start until AC-1 to AC-5 pass.

Q74. What span does a folded constant get?

The span of the entire expression it replaces (FR-O2), so the mapping still points at the original expression.

Q75. What is `--map-report` for?

It prints the share of source statements that still have at least one mapping, at `-O0` and `-O1`. Optimisation removes or merges code, so this measures how much debuggability is lost (goal G5, AC-6).

K. General compiler theory applied to Tether

Q76. Name the phases of a compiler and where Tether is now.

Lexical analysis, syntax analysis, semantic analysis, intermediate code (lowering to a core tree), optimisation, and code generation, with the symbol table and error handler shared across phases. Lexical analysis, syntax analysis and the error-reporting part of the error handler are complete.

Q77. What is the difference between a token, a lexeme and a pattern?

The token is the category (`Ident`), the lexeme is the actual text (`fib`), and the pattern is the rule that describes valid lexemes (a letter or _, then letters, digits or _). My token object stores kind, lexeme, decoded value and span.

Q78. What is the FIRST set of a statement in your grammar?

`let`, `if`, `while`, `for`, `return`, `print`, `{`, and everything that can start an expression: identifiers, integer and string literals, `true`, `false`, `(`, `-` and `!`. The alternatives are disjoint except that an identifier can start both an assignment and an expression statement, which is resolved by the second token.

Q79. Is your expression grammar ambiguous?

No. The stratified form fixes precedence and associativity, so every expression has exactly one parse tree, and precedence climbing implements the same rules.

Q80. Why doesn't the lexer use the symbol table?

Tether has no constructs, like C's `typedef`, where the meaning of a name changes how it is tokenised. Keywords are fixed, so the lexer stays context-free and the symbol table belongs entirely to semantic analysis.

Q81. What would you do differently if you started again?

Probably nothing structural. One option I considered was writing the expression grammar as Pratt parsing (null and left denotations), which would make adding postfix operators easier. Precedence climbing is equivalent for this operator set and shorter.

Appendix A. Five-minute live demo script

Run from the project folder. Node 18 or later is the only requirement.

1. Show that everything passes

```
$ npm test
```

Expected: 63 tests, 0 failures.

2. Tokens with spans (milestone 1 exit criterion)

```
$ node bin/tether.js build examples/fib.tth --dump-tokens
```

Expected: one token per line with line:col+length.

3. Syntax tree with spans (milestone 2 exit criterion)

```
$ node bin/tether.js build examples/fib.tth --dump-ast
```

Expected: indented tree, 83 nodes.

4. Many syntax errors in one run

```
$ node bin/tether.js build test/fixtures/syntax-errors.tth; echo "exit $?"
```

Expected: 7 errors with carets, exit 1.

5. Lexical errors without cascades

```
$ node bin/tether.js build test/fixtures/lex-errors.tth
```

Expected: 5 errors, no extra parser errors.

6. Honest about what is not built

```
$ node bin/tether.js build examples/fib.tth --dump-core; echo "exit $?"
```

Expected: "arrives with milestone 4", exit 2.

7. Playground

```
$ npm run playground   # open http://localhost:5173/playground/#fib
```

Expected: phase strip; hover nodes; delete a ";" and watch the error appear live; open #syntax-errors.

Appendix B. Glossary

Term	Meaning
Span	The exact source range of a token or node: file, offset, length, start and end line/column
AST	Abstract syntax tree: the program's structure without punctuation or helper grammar levels
Panic mode	Error recovery that skips tokens until a synchronising point (here, a statement boundary)
Precedence climbing	A table-driven, loop-based method of parsing binary operators with precedence and associativity
Lowering	Rewriting high-level constructs into simpler core constructs before code generation
Source Map v3	The standard JSON format that maps positions in generated code to positions in original source
Base64 VLQ	The variable-length number encoding used in the source-map `mappings` string
Golden file	A stored expected output that a test compares against byte for byte
Property test	A test that checks a rule holds for many randomly generated inputs
Cascade	Spurious follow-on errors caused by one real mistake